Tourism Explorer Component — Hackathon Problem Statement

1. The problem

When people plan a trip, they search broad questions like "things to do in Paris" or "places to visit in Kyoto". Answering well means understanding the intent and the place, finding real attractions nearby, filtering out restaurants, hotels and shops, and presenting what's worth seeing. Search results today mostly link out to long travel articles.

2. The challenge

Build a standalone hyperDart component that understands natural-language travel queries and helps someone decide what's worth seeing in a place.

At minimum, your component should handle:

- **Attractions in a place:** relevant sights, landmarks, museums and other points of interest near the searched city
- **Category requests:** queries that ask for one kind of place, such as museums or temples
- **Useful details for each place:** what it is, where it is, and a short description when one exists
- **No results:** a clear answer when nothing suitable is found

Apart from the basic requirements, you can add more features, like:

- **A day plan** that groups nearby places and suggests a sensible order
- **Weather for the visit day**, so people know whether to go
- **A map view** of the places, that also works in the narrow sidebar
- **Distance and directions** from the city centre or the user's location

> **What you show, how you show it, and how users interact with it are up to you. We're looking for original ideas, a unique design and solid engineering, not a copy of a travel site's listing page. The best components answer the question at a glance and then let people dig deeper.**

3. What we provide

Trigger keywords and regex

Add these to `hyperdart.config.js`. You may extend them. Places come from the `HD_LOCATION` collection type (cities, districts, states and countries). To trigger on other kinds of input too, add regexes using any collection from the Entities docs.

```
keywords: ['tourist attractions', 'things to do', 'places to visit',
'landmarks', 'sights', 'points of interest', 'museums']
```

```
regex: [
  '((best|top|famous)\\s+)?(tourist\\s+attractions?
|things\\s+to\\s+do|places\\s+to\\s+
(visit|see)|landmarks|sights|points\\s+of\\s+interest)\\s+
(in|near|around)\\s+HD_LOCATION(__\\w+)?.*',
  '((best|top|famous)\\s+)?(museums?|monuments?|temples?|forts?|viewpoints?
|parks?|historic(al)?\\s+sites?)\\s+
(in|near|around)\\s+HD_LOCATION(__\\w+)?.*'
]
```

searchData

Your component receives a `searchData` prop with the place found in the query, as an entity. Its coordinates are in `entityInfo.geo.lat` and `entityInfo.geo.long`. The category (museums, temples and so on) isn't resolved for you: work it out from `query`. Depending on the template, `searchData` may arrive as an array or a single object. Example for `Tourist attractions in NYC` (trimmed):

```
{
  "component": "@hyperdart/tourismexplorer",
  "query": "Tourist attractions in NYC",
  "entities": [
    {
      "word": "NYC",
      "collectionName": "CITIES",
      "collectionType": "HD_LOCATION",
      "entityInfo": {
        "geo": {
          "tz": "America/New_York",
          "lat": 40.71427,
          "city": "New York City",
          "long": -74.00597,
          "state": "New York",
```

```
          "country": "United States",
          "countryCode": "US"
        }
      },
      "entityType": "LOCATION",
      "description": "largest city in the United States",
      "relevanceScore": 100
    }
  ],
  "keyword": [{ "word": "tourist attractions", "description": "" }],
  "genericQuery": "tourist attractions in HD_LOCATION__CITIES",
  "userLocation": {
    "position": {
      "coords": { "latitude": 26.4631, "longitude": 80.3479 },
      "city": "Kanpur",
      "countryCode": "IN"
    },
    "source": "ip"
  },
  "suggestedPlace": "sidebar"
}
```

Data source

The Geoapify Places API. It needs a free API key: create a project at geoapify.com to get one.

```
GET https://api.geoapify.com/v2/places?
categories=tourism.sights&filter=circle:{longitude},
{latitude},5000&limit=20&apiKey=YOUR_API_KEY
```

- Coordinates go **longitude first**, then latitude.
- Request at most 20 places per call.
- **Keep your API key out of the published frontend.** Anything in frontend code is visible to every user, so call Geoapify from a backend (the React + Vite + Hono or Express templates).
- You may use other sources too, such as Wikipedia for descriptions or Open-Meteo for weather.

4. Example queries

User query	Component shows
`Tourist attractions in NYC`	Popular sights and landmarks around New York City
`Things to do in Paris`	Attractions worth visiting in Paris
`Landmarks in Rome`	Landmarks and historic sites first
`museums in Delhi`	Museums only
`best places to visit in Kurukshetra`	Places worth seeing in and around Kurukshetra
`Sights near Tokyo`	Sights around Tokyo

5. Submission

1. Source code in a public GitHub repository. Include a `.gitignore` so `node_modules` and `dist` aren't committed (the templates don't ship one). Never commit your API key.
2. The component published to the hyperDart dev registry.
3. A README covering your idea, design decisions, data sources, libraries, and how you built it (including any AI tools you used and what you wrote or changed yourself).
4. A 2–3 minute demo video on YouTube.

Add both links to `hyperdart.config.js`, using the field names already in your template:

```
github: 'https://github.com/<user>/<repo>',     // repo root, not a
/tree/main link
demovideo: 'https://www.youtube.com/watch?v=<id>'  // or
https://youtu.be/<id>
```

Publish with plain `npm publish`. It runs the prepublish validation (`scripts/validate-publish.js`), which checks keywords, regex, build output and both links. **Submissions published with validation skipped (for example `npm publish --ignore-scripts`, or by removing the `prepublishOnly` script) won't be evaluated.**

Before your first publish, create the package in the Partner Portal (My packages → Create package) and set your token with the command given in portal.

6. Judging criteria

Criterion	Weight	What judges look for
Design and creativity	25%	An original, unique design and interaction that works in sidebar and mainline, light and dark
Usefulness and clarity	20%	How quickly the component shows what's worth seeing in the place asked about
Accuracy and relevance	15%	Real tourist places near the right location; no restaurants, hotels or shops as primary results
Robustness	15%	Behaviour across different places, categories and queries, API failures and empty results
Understanding (live review)	15%	Explaining the code and making a small change live, for example adding a query pattern
Code quality and README	10%	Readable code, a clear README, and the API key kept out of the frontend

Extra features can lift a score, but only once the basic requirements work.

7. Resources

Resource	Link
hyperDart developer docs	hyperdart.com/docs/docs/intro
Templates (`npm create hyper-dart@latest`)	Templates
Keywords, regex and config	hyperdart.config.js
Integrating a backend (for your API key)	Integrating backend
Publishing	Publish
Entities (collection names and types for regexes)	Entities
Geoapify Places API	Places API docs
Geoapify categories	Category list